

Session 9 Lab

Inventory Management Program using Kotlin Collections & Generics

1. Objective

The objective of this lab is to develop a console-based Inventory Management System in Kotlin. The program uses collections such as lists and maps, along with generics and higher-order functions, to manage and filter inventory data. This exercise strengthens understanding of collection handling, reusable functions, and functional programming concepts in Kotlin.

2. Implementation

Data Class: Item

A data class is used to represent each inventory item with its name, category, and quantity.

```
data class Item(  
    val name: String,  
    val category: String,  
    val quantity: Int  
)
```

Function: printInventory()

This function prints items in a structured, table-like format.

```
fun printInventory(items: List<Item>) {  
    if (items.isEmpty()) {  
        println("No items found.")  
        return  
    }  
  
    for (item in items) {  
        println("Name: ${item.name} | Category: ${item.category} | Qty: ${item.quantity}")  
    }  
}
```

Generic Function: filterList()

This is a reusable generic function that filters any list based on a given condition.

```

fun <T> filterList(list: List<T>, condition: (T) -> Boolean): List<T> {
    val result = mutableListOf<T>()
    for (element in list) {
        if (condition(element)) {
            result.add(element)
        }
    }
    return result
}

```

This function works for any data type, making it flexible and reusable.

Main Function

The main function demonstrates:

- Creating inventory items
- Filtering low-stock items
- Filtering by category
- Calculating total quantity per category (Bonus)

```

fun main() {

    val inventory = listOf(
        Item("Laptop", "Electronics", 12),
        Item("Keyboard", "Electronics", 5),
        Item("Chair", "Furniture", 20),
        Item("Table", "Furniture", 8),
        Item("Notebook", "Stationery", 50),
        Item("Pen", "Stationery", 7)
    )

    println("--- Full Inventory ---")
    printInventory(inventory)

    // Filter low stock items (quantity < 10)
    val lowStockItems = filterList(inventory) { it.quantity < 10 }

    println("\n--- Low Stock (<10) ---")
    printInventory(lowStockItems)

    // Filter by category (Electronics)
    val electronicsItems = filterList(inventory) { it.category == "Electronics" }
}

```

```

println("\n--- Electronics Category ---")
printInventory(electronicsItems)

// BONUS: Total quantity per category
val totalsByCategory = inventory
    .groupBy { it.category }
    .mapValues { entry -> entry.value.sumOf { it.quantity } }

println("\n--- Total Quantity per Category ---")
for ((category, totalQty) in totalsByCategory) {
    println("$category: $totalQty")
}
}

```

3. Sample Output

```

--- Full Inventory ---
Name: Laptop | Category: Electronics | Qty: 12
Name: Keyboard | Category: Electronics | Qty: 5
Name: Chair | Category: Furniture | Qty: 20
Name: Table | Category: Furniture | Qty: 8
Name: Notebook | Category: Stationery | Qty: 50
Name: Pen | Category: Stationery | Qty: 7

```

```

--- Low Stock (<10) ---
Name: Keyboard | Category: Electronics | Qty: 5
Name: Table | Category: Furniture | Qty: 8
Name: Pen | Category: Stationery | Qty: 7

```

```

--- Electronics Category ---
Name: Laptop | Category: Electronics | Qty: 12
Name: Keyboard | Category: Electronics | Qty: 5

```

```

--- Total Quantity per Category ---
Electronics: 17
Furniture: 28
Stationery: 57

```

4. Concepts Practiced

This lab helped reinforce the following Kotlin concepts:

- Data classes for structured data representation
- Lists for storing collections of objects
- Maps for grouped data representation
- Generic functions for reusable logic
- Higher-order functions such as `groupBy` and `sumOf`
- Lambda expressions for filtering conditions

5. Reflection

Through this lab, I gained a deeper understanding of how Kotlin collections and generics work together to create reusable and maintainable code. Implementing a generic filter function improved my understanding of higher-order functions and lambda expressions. The use of grouping and aggregation functions demonstrated how powerful Kotlin's collection APIs are for handling real-world data scenarios efficiently.